

NVIDIA NOOA — Analyse Comparative

Source : [labs-OO-Agents](#) Paper : [arXiv 2607.20709](#) Date : Août 2026

NOOA en Bref

Concept central : un agent = un objet Python. Les méthodes avec `...` comme corps sont implémentées par un LLM au runtime, les méthodes normales restent du Python déterministe.

6 Capacités Clés

- 1. **Typed I/O** — Signatures avec annotations de type, validation avant retour
- 2. **Pass-by-reference** — Objets live injectés, pas serialisés en texte
- 3. **Code as action** — Le modèle écrit du Python dans un REPL Jupyter-style
- 4. **Programmable loop engineering** — Strategies (`Predict`, `CodeAct`) via décorateurs par méthode
- 5. **Explicit object state** — L'état est sur `self`, visible par le modèle
- 6. **Model-callable harness APIs** — `self.context`, `self.events` programmables

Architecture

Framework model-agnostic basé sur LiteLLM. Exécution sandboxée avec AST checks + deny-list. Mémoire long terme optionnelle via `MemoryManager`.

Résultats Benchmarks

SWE-bench Verified, Terminal-Bench 2.0, ARC-AGI-3. ~98% de réussite sur les tests de capacité.

Comparaison Détaillée

Dimension	NOOA	OpenHosta
Primitif central	Agent = classe Python, <code>...</code> = LLM	Agent = classe Python, <code>@compile()</code> + <code>@tool</code> , <code>@infer</code>
Méthodes agentic	Corps <code>...</code> détecté automatiquement	Décorateurs explicites : <code>@tool</code> , <code>@infer</code> , <code>@playbook</code> , <code>@planner</code> , <code>@router</code>
État	Sur <code>self</code> , direct	Sur <code>self</code> , avec système de lifecycle (CONFIGURED → RECRUITED → FREED)
Typage	Annotations Python natives + validation runtime	Guarded Types — couche sémantique avec validation, confiance, tolérance, origine, historique (CR-01 → CR-09)
Exécution code	REPL Jupyter-style, modèle écrit du Python	CapabilityDispatcher, routing vers des capacités déclarées

Dimension	NOOA	OpenHosta
Contexte	ContextManager + EventManager , statique/dynamique, événements typés	Traces, EventStream, TaskList, Workspace
Lifecycle	Absent	recruit/free/kill , quota tokens, états formalisés
Rôles & sécurité	Sandbox OS-level, AST deny-list	Roles/Authority , principal-based access control, Workspace confinement
Healing	Mécanisme dédié absent	Healer + HealingHook, récupération automatique
Multi-agent	Subagents via asyncio.gather	Downstream , AgentGraph, dépendances explicites
Mémoire	MemoryManager SQLite + embeddings + ACT-R activation	Non documenté dans V5 actuellement
Streaming	Absent	EventStream real-time
Scoping	self , method-scoped locals	Session API isolée, registres de capacités par session
Dépendances	LiteLLM, uv, Pydantic	Zéro dépendance (core), MIT

Points Communs

- L'agent comme objet Python
- Combinaison code déterministe + agentic
- Typage comme contrat
- Traces et observabilité

Avantages NOOA

- L'approche `...` est plus élégante que les décorateurs
- Le REPL Jupyter-style donne plus de contrôle au modèle
- Le pass-by-reference avec previews bornées est ingénieux (évite de surcharger le contexte, le prompt ne porte que des previews compactes)
- Mémoire long terme mature (MemoryManager avec SQLite, embeddings, activation ACT-R)
- Benchmarks publiés et reproductibles (SWE-bench Verified, Terminal-Bench 2.0, ARC-AGI-3)

Avantages OpenHosta

- **Guarded Types** — couche unique sans équivalent chez NOOA : validation + incertitude + tolérance + historique

- **Lifecycle formel** — recruit/free/kill avec quota de tokens, absent chez NOOA
- **Rôles/Authority** — contrôle d'accès principal-based, plus mature que le sandbox OS-level de NOOA
- **Healing** — récupération automatique, non présent chez NOOA
- **Architecture multi-agent** avec dependency graph explicite
- **Zéro dépendance** — plus léger, plus portable
- **5 types de capabilities** (`@tool`, `@infer`, `@playbook`, `@planner`, `@router`) vs 2 strategies chez NOOA (`Predict`, `CodeAct`)

En Résumé

NOOA est orienté **agent coding** : le modèle écrit du Python dans un REPL, le code est l'action. OpenHosta est orienté **enterprise** : lifecycle, sécurité, healing, multi-agent, guarded types.

Les deux convergent sur « *agent = objet Python* » mais avec des philosophies complémentaires.

Pistes d'Inspiration pour OpenHosta

Idée NOOA	Applicabilité à OpenHosta	Effort estimé
Méthode <code>...</code> comme méthode agentic	Intéressant, plus élégant que <code>@infer</code> , mais nécessite un moteur REPL Jupyter-style	Élevé
Pass-by-reference avec previews bornées	Utile pour les gros arguments ; l'actuel système de capabilities le fait déjà avec des schémas typés	Moyen
MemoryManager long terme	Manquant dans OpenHosta V5 ; l'approche ACT-R + SQLite est à étudier	Élevé
ContextManager/EventManager typés	L'EventStream d'OpenHosta couvre déjà le besoin ; les context blocks statiques/dynamiques sont une amélioration possible	Faible